

Inference Optimization

Inference Optimization is a technique used to make trained machine learning and generative models run faster, cheaper, and more efficiently in production.

Goal: The goal is to balance the "Golden triangle" of deployment: maximizing throughput while maintaining low latency and acceptable model quality.

These strategies scale across model-level compression and hardware-level infrastructure:

1. Model Compression & Architecture

- **Quantization:** Reduces the precision of model weights and activations (e.g., from 16-bit to 8-bit or 4-bit). This decreases memory footprint and increases processing speed without critically compromising output quality.
- **Distillation:** A process where a large, bulky "teacher" model trains a smaller, distilled "student" model to replicate its behavior with drastically fewer parameters.
- **Pruning:** Removes unnecessary or redundant parameters and connections within the neural network that contribute little to the final prediction

2. Runtime and memory management

- **KV Cache Optimization:** In Large Language Models (LLMs), the KV (Key-Value) cache stores past calculations for self-attention to avoid recomputing previous tokens. Frameworks like **PagedAttention** (used in vLLM) help manage this memory dynamically.
- **Speculative Decoding:** A technique where a smaller, faster "draft" model predicts multiple tokens at once, and a larger, slower model verifies them. This dramatically reduces token-generation latency.
- **Prefix Caching & Batching:** Reuses the Key-Value states of frequently used system prompts and groups multiple inference requests dynamically to ensure high GPU utilization

3. Infrastructure and Serving Strategies

- **Disaggregated Serving:** Separates the compute-bound "prefill" phase (processing the input prompt) from the memory-bound "decode" phase (generating the tokens), routing each to the hardware best suited for the job.
- **Hardware Acceleration:** Utilizes libraries like NVIDIA TensorRT-LLM to implement fused operations and highly optimized mathematical kernels tailored for GPUs.
- **Model Parallelism:** Splits massive models across multiple GPUs (Tensor Parallelism) or processes multiple requests across different servers (Pipeline Parallelism) to serve models that exceed single-GPU RAM